Report – IUM-TWEB

2024/2025

Project by: Cutrò Riccardo, Precisvalle Alessandro



UNIVERSITÀ
DI TORINO

Introduction:

In this report, we present IUM-TWEB project. The applications consists of a microservice architecture integrating multiple components, including a Java Spring Boot backend, two Express servers (node.js), two databases (PostgreSQL for static data and MongoDB for dynamic data) and a frontend component based on HTML5-CSS-JavaScript. The goal of this project is to create a movie platform for fans and journalists, to have access to data about films, which includes a large detailed movie list, and the possibility to search reviews written by critics and fans.

1) Backend: Java Spring Boot microservice

a) Solution:

We designed the main microservice in Spring Boot to handle movie data stored in PostgreSQL (static data). The Movie entity is mapped using JPA annotations to the relational database. For each table in the SQL Database we create “Model”, “Repository”, “Service” and “Controller”. The model is used to create the entity as a database table, with every its columns and type and getters and setters. The Controller handles requests from the Main Server: it receives incoming calls, processes them, and routes responses, querying the database through the Service. The Service in turn relies on the Repository, which executes the actual database queries. Once the data is retrieved, the Controller processes the response and sends it back to the Main Server. All related entities (Actors, Countries, Crews, Genres, Studios, Themes, etc.) are linked via custom queries using @Query annotations. All related entities have an auto-generated id to simplify their use, they also have a column id_movie to be referenced to movie.id (the_oscar_awards is referenced by the name of the film).

b) Issues

- Handling multiple relationships for aggregation, also using DTO (e.g. actors, crew, awards).
- Optimizing pagination and filtering queries.
- Optimizing efficiency (During development, we initially attempted to create composite primary keys, since entities other than *movies* did not have their own unique IDs. However, this approach proved to be inefficient, so we decided to switch to auto-generated IDs, which also provide a more scalable and efficient solution for future development)

c) Requirements

- Fully compliant with requirements for filtered REST APIs.
- Supports sorting, pagination, filtering by rating, release year, genre.
- Swagger implemented and fully operational.
- Efficient design ensures low latency API responses even under pagination.
- DTO aggregation covers advanced data representation.

d) Limitations

- Further scalability may require query optimizations or caching for very large datasets.
- Adding entirely new linked entities requires extending DTO and Repository interfaces.
- The filters are made by @Query annotations, they allow the speed of the Selects, but they are not dynamic.

2) Backend: NodeJS Express microservice (MongoDB)

a) Solution:

The reviews micro-service is implemented as a small Express application that communicates directly with a local MongoDB instance. A single Mongoose model, Reviews, defines every field that a review contains: movie title, critic name, review text, date, rating, and boolean flags such as top critic. Incoming HTTP requests are handled by an Express router mounted under the base path /reviews. Internally, each endpoint delegates to an asynchronous function that builds a MongoDB query object on the fly, also using filters like top_critic = true.

b) Issues:

- Designing flexible aggregation queries to support multiple filters combined (Top Critic, Review Type, Sort, Pagination).
- Ensuring consistent title matching between relational movie titles and review titles.

c) Requirements:

- Fully compliant with project requirements for flexible filtering of dynamic review data.
- Supports multi-filter search (Top Critic: true or false, Type: Fresh or Rotten, Date sort) via aggregation pipelines.
- Fully stateless REST API interface.
- Efficient pagination.

- Allows storing additional review fields without requiring schema changes.
- Easily extendable for future dynamic content.

d) Limitations:

- MongoDB pagination using skip may become inefficient with very high offsets on extremely large datasets.
- Data consistency depends on the accuracy of movie title matching across microservices.
- More complex aggregation pipelines may require optimization as dataset grows.

3) Backend: NodeJS Express Main server

a) Solution:

The Main Server acts as an API Gateway and orchestrator for the entire system. It is built using Node.js and Express, exposing a unified set of REST API endpoints that aggregate data from the backend microservices (Spring Boot for static movie data and MongoDB for reviews).
The Main Server handles incoming HTTP requests from the frontend, forwards them to the appropriate backend service, collects responses, merges them when needed, and returns unified results to the client.
It also handles cross-origin resource sharing (CORS). In addition, it manages WebSocket connections via Socket.IO to provide real-time chat functionality for the movie detail page. Each movie has a dedicated chat room identified by its movie ID, ensuring isolated live chats per movie detail page.

b) Issues:

- Coordinating and merging data from multiple independent microservices.
- Handling timeouts and errors gracefully when one of the backend services is temporarily unavailable.
- Managing stateful WebSocket communication (chat rooms) while keeping the main server stateless for HTTP requests.

- Ensuring consistency between frontend requests and backend service APIs.

c) Requirements:

- Fully functional API Gateway integrating multiple microservices.
- Routes all data (movies, reviews, chat) through a unified interface.
- Provides real-time chat functionality via Socket.IO namespace per movie.
- Implements robust error handling and proxy logic between microservices.
- Easily extendable for future microservices or additional functionalities.

d) Limitations:

- Being a central entry point, the main server could become a bottleneck if not scaled horizontally.
- Dependent on external services' availability (Spring Boot, MongoDB) for aggregated responses.
- WebSocket scaling may require external tools for production environments with many concurrent chat users.
-

4) Frontend: HTML + CSS + JAVASCRIPT

a) Solution:

The frontend is built using standard HTML5, CSS, JavaScript (vanilla), and Bootstrap for responsive design. It consists of double static HTML pages (index.html, details.html), where dynamic content is rendered after page load via client-side JavaScript logic. Each page communicates with the backend microservices through REST API calls and WebSocket (for real-time chat on details page), consuming data from both the Spring Boot microservice (for movie data) and the MongoDB microservice (for reviews).
The JavaScript modules handle API requests, filtering logic, pagination, sorting, rendering movie cards and details, and also manage WebSocket connections to enable live chat rooms per movie. Bootstrap ensures responsive design, while FontAwesome is used for rating stars and icons.

b) Issues:

- Managing multiple asynchronous API calls across services (Spring Boot, MongoDB, WebSocket).

- Handling fallback states (e.g., missing data, poster not available).
- Synchronizing pagination, filters, and dynamic updates.
- WebSocket integration required careful namespace separation to ensure proper room-based chat per movie.

c) Requirements:

- Fully responsive user interface with Bootstrap.
- REST API integration with filtering, sorting and pagination.
- WebSocket chat integrated with isolated rooms per movie.
- Modular and maintainable JavaScript code structure.
- Smooth user experience with loading indicators and error handling.

d) Limitations:

- Multi-page navigation requires full page reload between pages (not full client-side routing).
- No frontend framework therefore no state management or deep client-side routing.
- Error handling could be further improved with more robust feedback for API failures.

CONCLUSIONS:

In our opinion, this project has been one of the most challenging of the entire bachelor's program, especially this TWeb component. However, it has also been highly stimulating, as it allowed us to learn a wide range of concepts, techniques, and programming languages.
Throughout the development process, we encountered several implementation issues, all of which were successfully resolved, and for this reason we are very satisfied with the outcome.
Given the scope of the project, we sought to leverage our creativity and collaborated closely to bring it to completion. At this stage, the implemented features are still limited, but we have already laid a solid foundation for potential future enhancements.
In conclusion, we are fully satisfied with the results achieved.

DIVISION OF WORK:

Each team member contributed actively and significantly to the development of the project, fully meeting the requirements specified in the Assignment Document. Although the initial task allocation was not always strictly followed, the project was carried out in a highly collaborative manner.
Riccardo primarily worked on the Spring Boot server and the frontend, while Alessandro focused mainly on the MongoDB server and the main server. Nevertheless, both of us were involved in all parts of the project and contributed across all files and core components.

EXTRA INFORMATIONS:

To run and visualize the project, all servers must be started.
In order to correctly start the Spring Boot server, the following lines in the application.properties file need to be configured:
spring.datasource.password= your_Postgres_Password
spring.datasource.username= your_Postgres_Username
Then, set PostgresServerApplication as the main executable class. Upon execution, the tables will be automatically created in PGAdmin and must be populated with the cleaned CSV files.
For the Mongo microservice, it is necessary to connect to the MongoDB database, create the database, and populate the collections with the reviews data.

BIBLIOGRAPHY:

- OpenAI. *ChatGPT (Version 4.0)*. Language model used to assist with writing and editing during the project. OpenAI, 2025. Available at: https://openai.com/chatgpt
- W3Schools. *W3Schools Online Web Tutorials*. Retrieved June 25, 2025, from https://www.w3schools.com/